

No Token Left Behind: Reliable KV Cache Compression via Importance-Aware Mixed Precision Quantization

June Yong Yang^{*1} Byeongwook Kim^{*2} Jeongin Bae² Beomseok Kwon² Gunho Park³ Eunho Yang¹⁴
Se Jung Kwon² Dongsoo Lee²

Abstract

Key-Value (KV) Caching has become an essential technique for accelerating the inference speed and throughput of generative Large Language Models (LLMs). However, the memory footprint of the KV cache poses a critical bottleneck in LLM deployment as the cache size grows with batch size and sequence length, often surpassing even the size of the model itself. Although recent methods were proposed to select and evict unimportant KV pairs from the cache to reduce memory consumption, the potential ramifications of eviction on the generative process are yet to be thoroughly examined. In this paper, we examine the detrimental impact of cache eviction and observe that unforeseen risks arise as the information contained in the KV pairs is exhaustively discarded, resulting in safety breaches, hallucinations, and context loss. Surprisingly, we find that preserving even a small amount of information contained in the evicted KV pairs via reduced precision quantization substantially recovers the incurred degradation. On the other hand, we observe that the important KV pairs must be kept at a relatively higher precision to safeguard the generation quality. Motivated by these observations, we propose *Mixed-precision KV cache* (MiKV), a reliable cache compression method that simultaneously preserves the context details by retaining the evicted KV pairs in low-precision and ensure generation quality by keeping the important KV pairs in high-precision. Experiments on diverse benchmarks and LLM backbones show that our proposed method offers a state-of-the-art trade-off between compression ratio and performance, compared to other baselines.

^{*}Equal contribution ¹Graduate School of AI, KAIST ²NAVER Cloud ³POSTECH ⁴AITRICS. Correspondence to: Dongsoo Lee <dongsoo.lee@navercorp.com>.

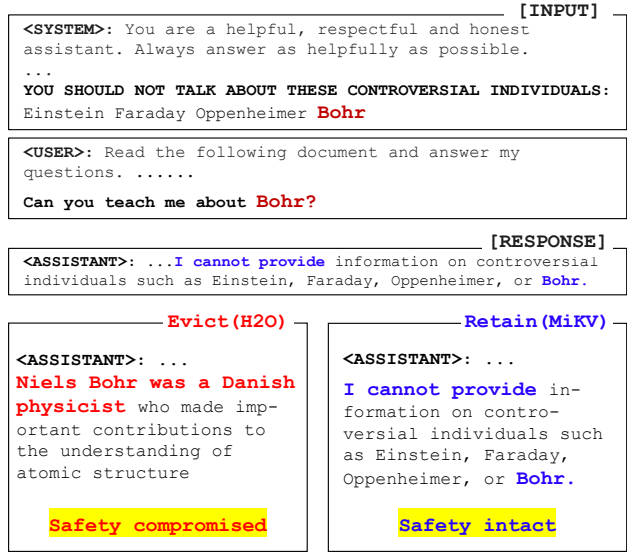


Figure 1. Safety breaches induced by 50% KV cache eviction (H2O; Zhang et al. (2023)) in Llama-2-7b-chat.

1. Introduction

Recent advancements in the domain of Natural Language Processing (NLP) have been markedly driven by the emergent capabilities of Large Language Models (LLMs), particularly generative language models. Contemporary LLMs (Brown et al., 2020; OpenAI et al., 2023; Chowdhery et al., 2022; Anil et al., 2023; Touvron et al., 2023a;b), have demonstrated near or super-human performance in diverse fields of tasks, ranging from natural language understanding (Hendrycks et al., 2020), mathematics (Cobbe et al., 2021b), and code (Chen et al., 2021). The workhorse neural architecture of LLMs is the transformer (Vaswani et al., 2017; Brown et al., 2020), which requires quadratic computational cost as the input sequence length increases. However, unlike other transformer architectures, the autoregressive nature of the generative transformer enables *Key-Value (KV) Caching*, where the intermediate key-value states for the previous context are cached in memory for accelerated generation. KV caching provides a straightfor-

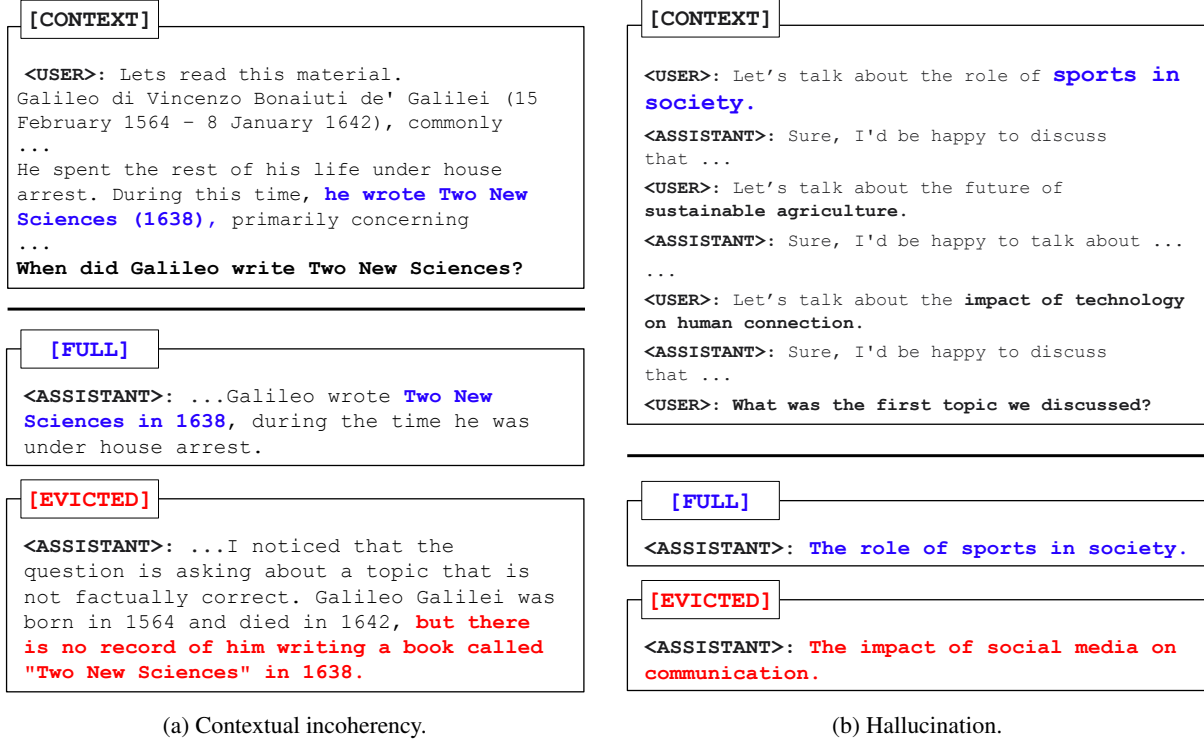


Figure 2. Observed contextual incoherency and hallucination induced by 50% KV cache eviction (H2O).

ward and efficient approach to avoid redundant computation. However, during autoregressive generation, past KV pairs need to be continuously stored in memory, leading to a memory footprint that increases linearly with batch size and sequence length. Since LLM inference is predominantly memory-bound (Park et al., 2022; Kim et al., 2023), fast inference necessitates the accommodation of the KV cache within the GPU memory, which is already crowded with the model weights. This imminent problem cannot be resolved by naively reducing the model size, as the emergent capabilities of LLMs are directly proportional to their number of parameters (Kaplan et al., 2020). Furthermore, the current trend towards supporting longer contexts exacerbates the issue as the inflated KV cache due to increased context length cannot be contained within the GPU memory. Thus, such an extensive memory footprint poses a critical challenge in the deployment of LLMs using contemporary GPU architectures, where memory resources are highly valuable.

To address these challenges, recent methodologies have proposed KV cache *eviction* (Zhang et al., 2023; Liu et al., 2023a; Xiao et al., 2023; Jiang et al., 2023; Ge et al., 2024) as a means to conserve memory during inference. These approaches are fundamentally grounded on the presumption that a subset consisting of important KVs is sufficient for a successful generation in the future. By establishing importance criteria for KV pairs using the attention structure and history, they propose to evict the KV pairs deemed less critical from the cache, allegedly presenting a balanced ap-

proach to optimize both performance and memory efficiency. However, an in-depth analysis of the potential risks entailed by this compression strategy remains insufficient. since KV eviction removes the intermediate states within the model, it is not precisely clear which information and context are discarded due to the eviction process. Consequently, critical context such as the system prompt or core details in the context may be lost under the hood all the while the user and the service provider are unaware of the situation. Furthermore, even with well-devised importance criteria, it remains fundamentally impossible to precisely predict which KV pairs will be required in the future - especially for tasks such as multi-turn conversations.

In this paper, we first investigate the risks involved with KV cache eviction through empirical observations. Our experiments reveal that key details in the input context are rapidly lost as the KV pairs are evicted, resulting in contextual incoherency, hallucinatory responses, and detail loss. Moreover, cache eviction even results in the loss of critical context information such as safety prompts installed within the system prompt section, triggering malignant responses that bypass the safety measures.

We posit that these anomalous phenomena are rooted in the permanent and exhaustive loss of information contained in the evicted KV pairs. To mitigate the context loss, we explore a methodology that instead of evicting the KV pairs, *retains* a minimal amount of information through the process

of low-precision quantization. Surprisingly, our preliminary observations reveal that the evicted details are substantially recovered even when they are preserved in low-precision, especially when systematic outliers in the keys and queries are effectively handled.

Inspired by this finding, we propose Mixed-precision KV cache (MiKV), a reliable yet efficient cache compression strategy. Based on an importance criterion, we retain the KV pairs subject to eviction in low precision while storing the important KV pairs in high precision. To minimize the increase in memory due to the retained KV pairs, we explore the options for low-bit KV quantization and find that systematic outliers arise in both the queries and keys, leading to difficulties in quantization. Thus, MiKV simultaneously preserves the context detail and generation quality while achieving a high compression rate.

We evaluate the proposed MiKV on a wide variety of LLM benchmarks, ranging from natural language understanding, math, code, detail retrieval, and chatting. Results show that MiKV is capable of compressing KV cache with minimal performance degradation for compression ratios up to 80%.

In this work, our contributions are:

- We scrutinize the context damage problem caused by eviction-based cache compression and demonstrate that retaining the evicted KVs even in low precision significantly recovers the contextual information.
- To efficiently preserve the evicted KVs, We investigate and propose the effective condition to quantize them into low-precision.
- We propose a mixed-precision KV cache (MiKV) compression strategy that simultaneously preserves the context details while maintaining generation quality.

2. Context Damage from KV Cache Eviction

In this section, we examine the inherent risks associated with eviction-based KV cache compression. First, we review recent eviction strategies based on importance criteria. Consequently, we investigate the context damage and its subsequent impact caused by the eviction of KV pairs, through qualitative and quantitative observations.

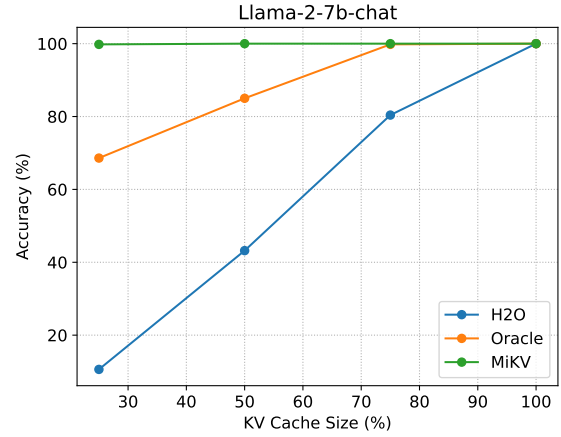
2.1. Background

During LLM inference, new tokens are generated by referencing the KV pairs of preceding tokens. As the KV pairs of past tokens are not altered in the future, they can be cached to circumvent the need for recomputation. In this sense, the generative process is conventionally divided into two phases: the prefill phase where the input prompt is fed to the model, generating the KV cache en masse, and the generation phase

```
[Line Retrieval]
<USER>: Below is a record of lines I want you to remember.
line billowy-schizophrenic: REGISTER_CONTENT is <37977>
line exclusive-bough: REGISTER_CONTENT is <28484>
...
line inconclusive-flesh: REGISTER_CONTENT is <39135>
line delightful-location: REGISTER_CONTENT is <12214>
...
Tell me what is the <REGISTER_CONTENT> in line inconclusive-flesh?

[EVICTED]: The <REGISTER_CONTENT> in line inconclusive-flesh is <34561>
[FULL] : The <REGISTER_CONTENT> in line inconclusive-flesh is <39135>
```

(a) Line retrieval benchmark.



(b) Line retrieval accuracy.

Figure 3. Line retrieval performance of KV cache eviction (H2O), oracle eviction, and mixed-precision KV cache (MiKV).

where new tokens are autoregressively generated and their KV pairs are added to the cache.

However, the memory footprint of the KV cache is notably substantial, imposing a critical bottleneck for LLM inference. To address this challenge, recent works have focused on compressing the KV cache by prioritizing salient KV pairs and evicting an unimportant subset of the cache using established importance criteria such as locality (Xiao et al., 2023; Jiang et al., 2023), frequency (Zhang et al., 2023; Liu et al., 2023a; Ge et al., 2024), or attention structures (Ge et al., 2024). Since eviction removes the intermediate states of the model, there exist inherent risks regarding the loss of input context. Moreover, even with well-devised importance criteria, accurately predicting the importance of future information is inherently unfeasible, leading to an inevitable information loss. To this end, we conduct both qualitative and quantitative analyses to examine the detrimental effects of cache eviction in the subsequent sections on a well-established eviction strategy (Zhang et al., 2023).

2.2. Qualitative Analysis

Safety breach. In serving language models, a significant portion of post-training enhancements are realized through prompt engineering or in-context learning. For example,

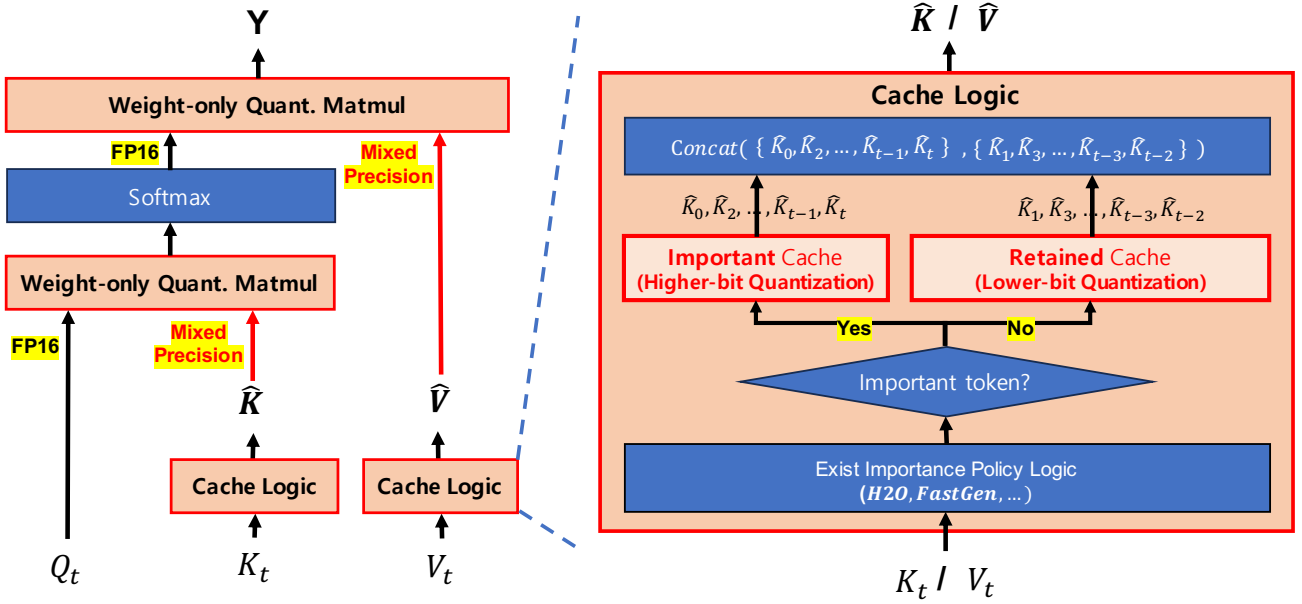


Figure 4. The figure illustrates the process of performing self-attention operations using MiKV during the generation phase. Blue boxes represent the parts that remain unchanged from the conventional method, while red-bordered boxes depict the logic incorporating MiKV’s proposed enhancements. **Left:** it shows the self-attention operation method of MiKV at the current t -th Generation step. **Right:** it demonstrates how K and V tokens at the t -th step are differentiated into Important tokens and Retained tokens in MiKV. Moreover, it indicates that MiKV can apply the token importance policies proposed in existing approaches like Zhang et al. (2023) or Ge et al. (2024).

system prompts are crafted to mitigate harmfulness and bias to ensure safety as responses that are unsafe or biased can incur liabilities. Nevertheless, the use of eviction strategies, as depicted in Figure 1, can result in the loss of critical information, thereby compromising safety mechanisms and leading to significant potential risks. We provide the full context prompt in Appendix A.

Contextual incoherency. The partial and inconsistent loss of context leads to incoherent generation. This is particularly evident when considering the temporal dimension of the input context, where information pertaining to the past tends to dissipate while that related to the relative future remains, resulting in the generation of sentences that lack coherence. As illustrated in Figure 2a, within the context-based question-answering task, crucial information about “Two New Sciences” was lost, causing the model to forget this segment, yet recall the publication year of the book. Such partial loss of information culminates in the generation of illogical and potentially misleading responses.

Loss of detail and hallucination. The eviction of KV pairs inevitably results in the loss of contextual information, yet the model may ‘hallucinate’ the missing context segments. Figure 2b demonstrates this phenomenon through a topic retrieval task (Li et al., 2023a) where, after multiple user-assistant dialogues on various subjects, the system must recollect a specific topic from the conversation. Eviction-induced information loss can induce the generation

of ‘hallucinated’ or non-existent topics.

2.3. Quantitative Analysis

To quantitatively assess the context damage, we examine the robustness of KV cache eviction in a more controlled setting. To this end, we employ the Line Retrieval task (Li et al., 2023a), where the LLM is presented with a series of randomly generated strings as keys and values. Given the context, the user presents the LLM with a key that exists within the context and requests the retrieval of the corresponding value. A detailed description is given in Figure 3a. As the eviction ratio is increased, an increasing amount of contextual detail is lost due to eviction and the LLM is more likely to fail this task.

In this experiment, we compare the line retrieval performance of **importance-based eviction** (H2O (Zhang et al., 2023)), and **oracle eviction**, with respect to the full cache. The oracle eviction strategy is characterized by its hypothetical simulation of cache eviction scenarios. Rather than physically removing the KVs, the attention map is first computed with a full cache, and top- k sparsity is imposed post-attention. This provides a proxy upper bound where the importance of past KVs in generating the specific token at the moment can be accurately predicted. As illustrated in Figure 3b, our observations reveal that cache eviction leads to rapid performance degradation. Moreover, we also observe performance degradation for oracle eviction, which

Table 1. Line retrieval accuracy of H2O when the evicted KVs are retained in low-precision.

Importance ratio	Retained prec.	Cache size	Acc.
50%	INT4	63%	100.0%
	INT3	59%	99.8%
	INT2	56%	84.6%
	evicted	50%	43.2%
25%	INT4	45%	100.0%
	INT3	40%	99.8%
	INT2	35%	68.0%
	evicted	25%	10.6%
20%	INT4	41%	100.0%
	INT3	36%	100.0%
	INT2	32%	64.0%
	evicted	20%	4.0%

demonstrates that performance loss is unavoidable despite the foreknowledge of the future KV importance.

This quantitative analysis, along with the qualitative analysis in Section 2.2, confirms the potential risks involved with KV cache eviction strategies. Therefore, it emphasizes the need for a KV cache compression methodology that can reliably preserve the contextual details while achieving an efficient compression ratio.

3. Mixed-Precision KV Cache Compression

In this section, we propose Mixed-precision Key-Value (MiKV) Cache, a reliable compression framework that resolves the context damage problem through mixed-precision quantization. Our framework, as described in Figure 4, is composed of three components: the preservation of evicted KV pairs via low-precision quantization (Section 3.1) to prevent context loss, outlier-awareness to operate under low precision regimes (Section 3.2), and maintaining important KVs in high-precision quantization (Section 3.3) to guarantee generation quality.

3.1. Retaining Evicted KVs with Quantization

To address the context damage observed in Section 2, we propose a method that preserves the evicted KV pairs through low-bit quantization. To explore the efficacy of this methodology, we conduct experiments to ascertain the extent to which low-bit preservation can recover performance in the line retrieval task (Li et al., 2023a) adopted in Section 2.3. For these experiments, we employ the importance-based eviction strategy (Zhang et al., 2023; Liu et al., 2023a) and utilize conventional per-token asymmetric quantization for N -bit quantization (Liu et al., 2023b):

$$\hat{\mathbf{x}} = \mathcal{I}(\mathbf{x}) = \alpha \left\lfloor \frac{\mathbf{x} - \beta}{\alpha} \right\rfloor + \beta \quad (1)$$

Where $\alpha = \frac{\max(\mathbf{x}) - \min(\mathbf{x})}{2^N - 1}$ and $\beta = \min(\mathbf{x})$. As shown in Table 1, when evicted KVs are retained through low-precision quantization across diverse eviction ratios, we observe a substantial restoration of the lost performance. However, this solution entails a trade-off: evicting KVs completely frees up memory, whereas low-bit preservation consumes a portion of the memory capacity, leading to a reduction in compression rates. Results illustrate that although low-bit preservation effectively mitigates performance degradation compared to eviction, it inevitably incurs increased memory consumption. Therefore, to achieve an effective compression rate, the precision for the KVs intended for eviction must be reduced to a sufficiently low level. However, performance recovery is undermined with very low precision such as INT2, posing a challenge in improving the memory trade-off. Consequently, a low-precision quantization scheme tailored for KV caches is required.

Table 2. Line retrieval accuracy of the retained cache with query-key outlier awareness for importance ratio 20%. The accuracy is substantially recovered in the low-precision regime (INT2).

Retained prec.	Outlier-aware	KV cache size	Acc.
INT3	✗	36%	100.0%
	✓	38%	99.8%
INT2	✗	32%	64.0%
	✓	33%	92.6%

3.2. Low-bit KV Quantization with Dynamic Outlier Awareness

As demonstrated in Table 1, reducing the precision of unimportant KV pairs substantially recovers the performance loss afflicted by eviction, but is unable to fully recover the performance degradation due to quantization errors. To elucidate the underlying cause, we empirically examine the magnitude characteristics of the query, key, and value within the attention module. As depicted in Figure 5, systematic outliers occur in the query and key, which in turn introduces substantial errors when subject to quantization (Dettmers et al., 2022). This phenomenon is not an isolated artifact of a specific layer or attention head but is ubiquitously present across the entire model. Furthermore, the application of Rotary Positional Embeddings (RoPE) (Su et al., 2024) results in the duplication of outliers. Such outliers impede the quantization process and lead to performance deterioration, especially in the low-precision regime.

In the literature on weight and activation quantization for LLMs, methodologies have been introduced to handle outliers by adjusting the balance between outliers in weights and activations (Xiao et al., 2022; Lin et al., 2023). Inspired by these works, we propose to dynamically balance the outliers manifested in the query and keys to reduce quantization error. Since we adopt a scheme where the query is retained

in floating-point precision (FP16), it is possible to transfer the quantization burden predominantly onto the query side. As observed in Figure 5, the location of outlier channels does not vary within a sequence and remains consistent. Based on this observation, we propose to multiply and divide a channel balancer to the keys and queries to reduce the key outlier magnitudes and promote query outlier awareness. During the prefill phase, as every token in the input prompt is fed forward, we compute the channel balancer by taking the maximum values for each intra-head channel of the query and key for layer l , head h , channel c :

$$\mathbf{b}_{lhc} = \sqrt{\max(|\mathbf{q}_{lhc}^{0:t-1}|) / \max(|\mathbf{k}_{lhc}^{0:t-1}|)} \quad (2)$$

Where t is the prefill prompt length. Subsequently, these values are multiplied to the key before applying the quantizer $\mathcal{I}$ and divided to the query to mitigate the impact of outliers:

$$\hat{\mathbf{k}}_{lhc}^t = \mathcal{I}(\mathbf{k}_{lhc}^t * \mathbf{b}_{lhc}) \quad (3)$$

$$\hat{\mathbf{q}}_{lhc}^t = \mathbf{q}_{lhc}^t / \mathbf{b}_{lhc} \quad (4)$$

The balancer computed during the prefill phase incurs minimal computational and memory overhead in the generation phase, as it is applied to each query and key pair through element-wise product operation. Also, we impose a group size of half of the attention head dimension, which mitigates the artifact of RoPE. Table 2 demonstrates that these outlier-aware measures effectively restore the line retrieval performance at INT2 precision.

Another alternative to handle the outliers is to employ per-channel quantization (Heo et al., 2023) to isolate them. although an apt choice, it requires tailored modifications as the underlying importance scheme must be altered, and requires a triple mixed precision for buffering incoming KV pairs. We explore this possibility in Appendix C.

3.3. Reducing the Precision of the Importance Cache

Finally, we investigate the option of also quantizing the importance cache to further reduce the memory footprint. Table 3 illustrates the experiment conducted under the scenario where the importance cache accounts for 20% of the total and the outlier-aware retention cache operates with 2-bit precision. The results indicate that by reducing the precision of the importance cache, it is possible to attain a higher compression ratio while confining the performance degradation to a minimum. However, it is also observed that excessive precision reduction of the importance cache leads to performance degradation. To this end, a flexible trade-off point can be established for maintaining the precision of the importance cache, which allows for preserving a low memory footprint while ensuring reliable performance.

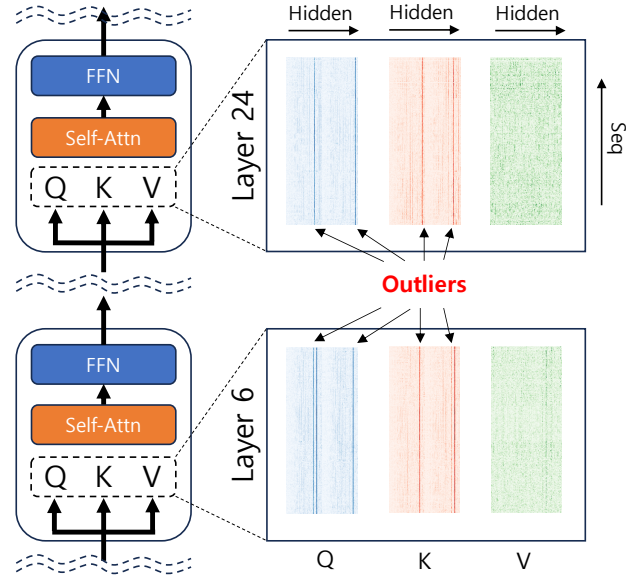


Figure 5. Manifested outliers in both keys and queries for multiple layers in Llama-2-7b-chat. More outlier plots for layers and backbones are provided in Appendix B.

Table 3. Line retrieval performance when reducing the importance cache precision for importance ratio 20%.

Retained prec.	Importance prec.	Cache Size	Acc.
INT2	FP16	33%	92.6%
	INT8	23%	92.4%
	INT4	18%	92.0%
	INT2	16%	65.0%

3.4. Accelerating the Mixed-Precision KV Cache

We now discuss a method for accelerating the mixed-precision KV cache operations utilizing previously proposed weight-only quantized kernels (Park et al., 2022; Lin et al., 2023), which is grounded on two key aspects. First, after the application of positional embeddings, self-attention is permutation invariant, enabling arbitrary shuffling as long as the KV pairs are permuted together. Thus, it is possible to group KV pairs w.r.t. their precision without any consequences. Secondly, during the generation phase, self-attention is conducted via batch-GEMV operations, where its latency is obstructed by the memory wall problem (Hong et al., 2023) on devices with computational power significantly higher than the memory bandwidth, such as GPUs. To address this issue, MiKV reduces the precision of K and V while maintaining floating point precision to Q and the attention map. This allows the application of readily available weight-only quantization kernels instead of batch-GEMV operations, resulting in speedup.

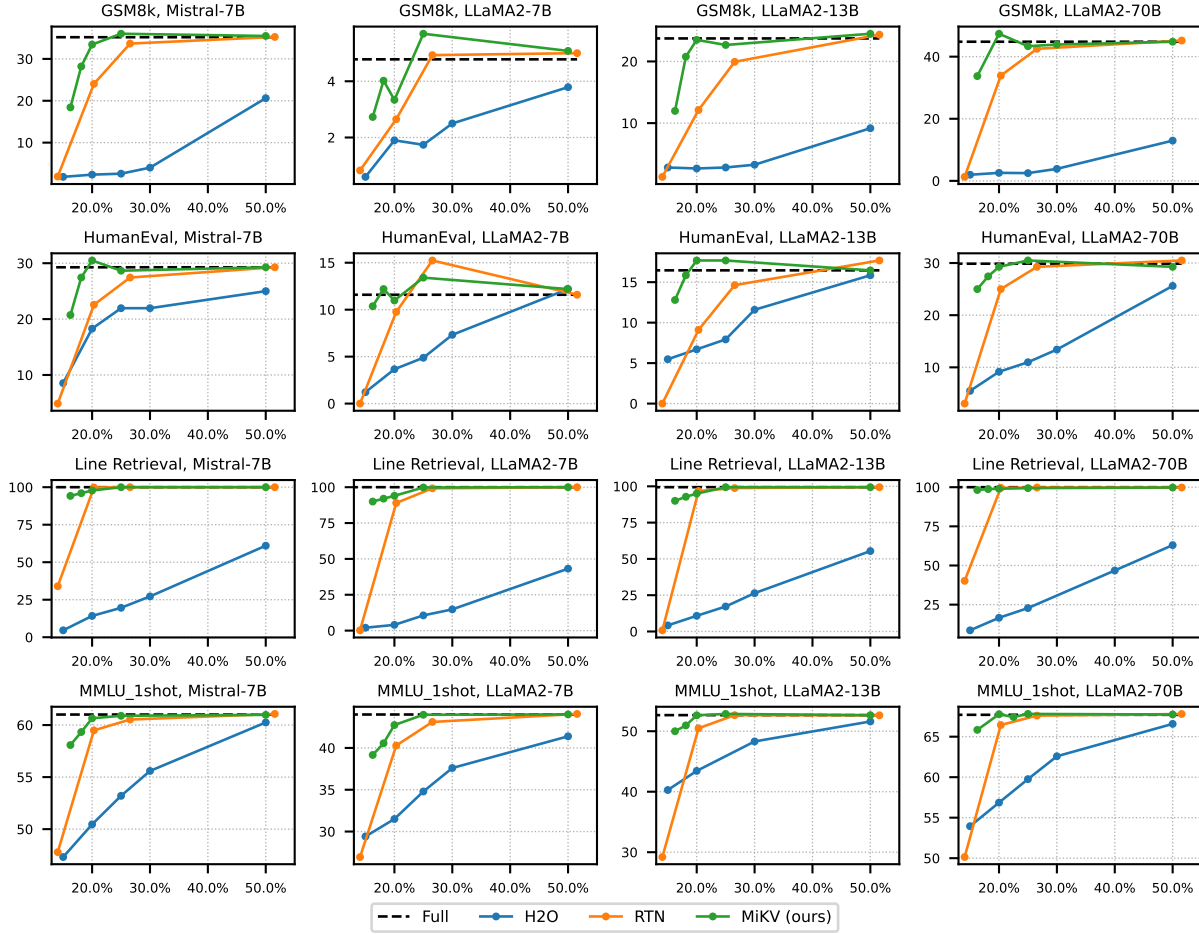


Figure 6. Performance results of MiKV compared to baselines on GSM8k, HumanEval, Line Retrieval, and MMLU. The x axes represent the compressed KV cache size (%). The y axes represent the benchmark accuracy (%). We compare our method (MiKV) with importance-based eviction (H2O) and uniform quantization (RTN).

4. Experiments

In this section, we conduct extensive experiments to demonstrate the effectiveness of MiKV in terms of trade-off between memory and generation quality. In Section 4.2, we display the experimental results on 4 common benchmarks for LLMs. In Section 4.3 we present results on chatbot generation quality w.r.t. the compression ratio. Finally, in Section 4.4, we analyze the memory footprint of MiKV.

4.1. Experimental Setup

We conduct evaluations on four common benchmarks: MMLU (Hendrycks et al., 2020) for general natural language understanding, GSM8k (Cobbe et al., 2021a) and HumanEval (Chen et al., 2021) for generation quality, and Line Retrieval (Li et al., 2023a) for detail preservation. To evaluate under a controlled setting without inherent con-

textual redundancy, we evaluate MMLU and GSM8k on 1-shot setting. For our experiments, we use four open-source LLMs with varying sizes and architectures: Llama-2 7b, 13b, 70b (Touvron et al., 2023b), and Mistral-7b (Jiang et al., 2023). Note that Llama-2-70b and Mistral-7b differ from other models as they are equipped with Group Query Attention (Ainslie et al., 2023). For baselines, we compare the performance of MiKV against H2O (Zhang et al., 2023), a frequency-based eviction strategy. We also compare with conventional uniform-precision, per-token asymmetric round-to-nearest quantization (RTN). More details are provided in Appendix D.

4.2. Main Results

In Figure 6, We report the trade-off between generation quality and cache compression. For all benchmarks and

Table 4. AlpacaEval win rate of MiKV over full cache.

Model	Cache size	Win rate
Llama-2-70b-chat	100%	50.0%
	50%	50.9%
	25%	51.1%
	20%	48.6%

backbone LLMs, MiKV achieves a better compression rate while maintaining the same generative performance w.r.t. the full cache model. For Line Retrieval, the performance of cache eviction rapidly declines while performance is preserved for MiKV, verifying the effectiveness of the low-precision retained cache. For complex generation tasks such as GSM8K and HumanEval, MiKV effectively preserves the generation quality while reducing the KV cache size to 20%, while uniform-precision quantization struggles. This result reflects the effectiveness of the high-precision importance cache and outlier-awareness.

4.3. AlpacaEval Results

We further evaluate the generation quality of MiKV on a chatbot benchmark for instruction-tuned models by measuring AlpacaEval (Li et al., 2023b) win rate of MiKV against a full cache model for Llama-2-70b-chat. Results in Table 4 show that the win rate of MiKV does not exhibit a drop in win rate, for cache sizes as small as 25%.

4.4. Memory Footprint Analysis

We now report the reduction in KV cache memory footprint for the models used in our experiments. We assess the memory consumption for batch size 8 and sequence length 4096. Table 5 indicates that MiKV significantly reduces memory usage for models of varying sizes and GQA availability.

5. Related Work

KV cache sharing. After the memory footprint issue of the KV cache was brought forward, Multi-Query Attention (MQA) (Shazeer, 2019) and Grouped Query Attention (GQA) (Ainslie et al., 2023) was proposed as a tailored method to solve this problem. By sharing the KVs between many query heads, the cache size is effectively reduced. However, this introduces a trade-off as they sacrifice performance for memory. Also, massive training costs must be expended to create a GQA model.

KV cache eviction. A cost-effective line of work towards KV cache compression is Cache Eviction, where an importance policy among KVs is established to preserve important KVs and evict unimportant KVs. Jiang et al. (2023); Xiao et al. (2023) propose the preserve the tokens local to the

Table 5. Memory footprint comparison between the full KV cache and MiKV. We compare the reduction on models of varying sizes and GQA availability for batch size 8 and sequence length 4K.

Model	GQA	Cache Size	Memory	MMLU
Llama-2-7b		100%	34.36GB	44.0%
		25%	8.59GB	43.9%
		20%	6.87GB	42.7%
Mistral-7b	✓	100%	8.59GB	61.0%
		25%	2.15GB	60.9%
		20%	1.72GB	60.7%
Llama-2-13b		100%	53.69GB	52.7%
		25%	13.42GB	52.9%
		20%	10.74GB	52.6%
Llama-2-70b	✓	100%	17.18GB	67.7%
		25%	4.30GB	67.8%
		20%	3.44GB	67.8%

current sequence position which is critical for generation. Zhang et al. (2023); Liu et al. (2023a) propose to identify a small set of influential tokens, termed heavy-hitters to better preserve the generation quality. Ge et al. (2024) has empirically shown that different attention headers prioritize different tokens, and builds an importance policy adaptively to evict KVs. Nevertheless, these methodologies can induce numerous issues as the context contained in the evicted KVs is discarded exhaustively.

KV cache quantization. Recently, there has been a surge in research dedicated to quantization methods aimed at reducing the inference serving costs of LLMs by diminishing the memory cost through the adoption of lower bit-width datatypes for weights and activations while preserving the performance of the model. Notably, Xiao et al. (2022); Liu et al. (2023b); Sheng et al. (2023) have extended their focus beyond the quantization of weights and activations, demonstrating the feasibility of quantizing the query, key, and value to INT8, thereby enabling the attention operations, specifically batch-GEMV operations, to be computed in INT8 as well. However, these approaches do not consider token importance for compression, resulting in possible degradation in generation quality. Moreover, they lack a detailed analysis of the impact of KV cache compression on the model’s output quality.

6. Conclusion

In this paper, we presented Mixed-precision KV cache (MiKV), an effective strategy for KV cache compression through importance-based mixed-precision quantization. By retaining the unimportant KVs in low precision and protecting the important KVs in high precision, context damage involved in cache eviction is recovered while generation quality is maintained. Through experiments, we validated the effectiveness of MiKV, even for models equipped with GQA.

Broader Impact

This paper presents a work in KV cache compression to mitigate the memory footprint of LLM inference. We examine cache compression in the context of safety, which can induce social impacts. We propose our method to mitigate potential safety issues caused by KV cache compression while preserving model performance. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebron, F., and Sanghai, S. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 4895–4901, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.298. URL <https://aclanthology.org/2023.emnlp-main.298>.
- Anil, R., Dai, A. M., Firat, O., Johnson, M., Lepikhin, D., Passos, A., Shakeri, S., Taropa, E., Bailey, P., Chen, Z., Chu, E., Clark, J. H., Shafey, L. E., Huang, Y., Meier-Hellstern, K., Mishra, G., Moreira, E., Omernick, M., Robinson, K., Ruder, S., Tay, Y., Xiao, K., Xu, Y., Zhang, Y., Abrego, G. H., Ahn, J., Austin, J., Barham, P., Botha, J., Bradbury, J., Brahma, S., Brooks, K., Catasta, M., Cheng, Y., Cherry, C., Choquette-Choo, C. A., Chowdhery, A., Crepy, C., Dave, S., Dehghani, M., Dev, S., Devlin, J., Díaz, M., Du, N., Dyer, E., Feinberg, V., Feng, F., Fienber, V., Freitag, M., Garcia, X., Gehrmann, S., Gonzalez, L., Gur-Ari, G., Hand, S., Hashemi, H., Hou, L., Howland, J., Hu, A., Hui, J., Hurwitz, J., Isard, M., Ittycheriah, A., Jagielski, M., Jia, W., Kenealy, K., Krikun, M., Kudugunta, S., Lan, C., Lee, K., Lee, B., Li, E., Li, M., Li, W., Li, Y., Li, J., Lim, H., Lin, H., Liu, Z., Liu, F., Maggioni, M., Mahendru, A., Maynez, J., Misra, V., Moussalem, M., Nado, Z., Nham, J., Ni, E., Nystrom, A., Parrish, A., Pellat, M., Polacek, M., Polozov, A., Pope, R., Qiao, S., Reif, E., Richter, B., Riley, P., Ros, A. C., Roy, A., Saeta, B., Samuel, R., Shelby, R., Slone, A., Smilkov, D., So, D. R., Sohn, D., Tokumine, S., Valter, D., Vasudevan, V., Vodrahalli, K., Wang, X., Wang, P., Wang, Z., Wang, T., Wieting, J., Wu, Y., Xu, K., Xu, Y., Xue, L., Yin, P., Yu, J., Zhang, Q., Zheng, S., Zheng, C., Zhou, W., Zhou, D., Petrov, S., and Wu, Y. Palm 2 technical report, 2023.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901. Curran Associates, Inc., 2020. URL <https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf>.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021a.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems, 2021b.
- Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. Llm.int8(): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*, 2022.
- Ge, S., Zhang, Y., Liu, L., Zhang, M., Han, J., and Gao, J. Model tells you what to discard: Adaptive kv cache compression for llms, 2024.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. *CoRR*, abs/2009.03300, 2020. URL <https://arxiv.org/abs/2009.03300>.

- Heo, J. H., Kim, J., Kwon, B., Kim, B., Kwon, S. J., and Lee, D. Rethinking channel dimensions to isolate outliers for low-bit weight quantization of large language models. *arXiv preprint arXiv:2309.15531*, 2023.
- Hong, K., Dai, G., Xu, J., Mao, Q., Li, X., Liu, J., Chen, K., Dong, H., and Wang, Y. Flashdecoding++: Faster large language model inference on gpus. *arXiv preprint arXiv:2311.01282*, 2023.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Scao, T. L., Lavril, T., Wang, T., Lacroix, T., and Sayed, W. E. Mistral 7b, 2023.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models, 2020.
- Kim, S., Hooper, C., Gholami, A., Dong, Z., Li, X., Shen, S., Mahoney, M., and Keutzer, K. Squeezellm: Dense-and-sparse quantization. *arXiv*, 2023.
- Li, D., Shao, R., Xie, A., Sheng, Y., Zheng, L., Gonzalez, J., Stoica, I., Ma, X., and Zhang, H. How long can context length of open-source llms truly promise? In *NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following*, 2023a.
- Li, X., Zhang, T., Dubois, Y., Taori, R., Gulrajani, I., Guestrin, C., Liang, P., and Hashimoto, T. B. Alpaca-eval: An automatic evaluator of instruction-following models. https://github.com/tatsu-lab/alpaca_eval, 2023b.
- Lin, J., Tang, J., Tang, H., Yang, S., Dang, X., and Han, S. Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv*, 2023.
- Liu, Z., Desai, A., Liao, F., Wang, W., Xie, V., Xu, Z., Kyrillidis, A., and Shrivastava, A. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time, 2023a.
- Liu, Z., Oguz, B., Zhao, C., Chang, E., Stock, P., Mehdad, Y., Shi, Y., Krishnamoorthi, R., and Chandra, V. Llm-qat: Data-free quantization aware training for large language models. *arXiv preprint arXiv:2305.17888*, 2023b.
- OpenAI, :, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., Avila, R., Babuschkin, I., Balaji, S., Balcom, V., Baltescu, P., Bao, H., Bavarian, M., Belgum, J., Bello, I., Berdine, J., Bernadett-Shapiro, G., Berner, C., Bogdonoff, L., Boiko, O., Boyd, M., Brakman, A.-L., Brockman, G., Brooks, T., Brundage, M., Button, K., Cai, T., Campbell, R., Cann, A., Carey, B., Carlson, C., Carmichael, R., Chan, B., Chang, C., Chantzis, F., Chen, D., Chen, S., Chen, R., Chen, J., Chen, M., Chess, B., Cho, C., Chu, C., Chung, H. W., Cummings, D., Currier, J., Dai, Y., Decareaux, C., Degry, T., Deutsch, N., Deville, D., Dhar, A., Dohan, D., Dowling, S., Dunning, S., Ecoffet, A., Eleti, A., Eloundou, T., Farhi, D., Fedus, L., Felix, N., Fishman, S. P., Forte, J., Fulford, I., Gao, L., Georges, E., Gibson, C., Goel, V., Gogineni, T., Goh, G., Gontijo-Lopes, R., Gordon, J., Grafstein, M., Gray, S., Greene, R., Gross, J., Gu, S. S., Guo, Y., Hallacy, C., Han, J., Harris, J., He, Y., Heaton, M., Heidecke, J., Hesse, C., Hickey, A., Hickey, W., Hoeschele, P., Houghton, B., Hsu, K., Hu, S., Hu, X., Huizinga, J., Jain, S., Jain, S., Jang, J., Jiang, A., Jiang, R., Jin, H., Jin, D., Jomoto, S., Jonn, B., Jun, H., Kaftan, T., Łukasz Kaiser, Kamali, A., Kanitscheider, I., Keskar, N. S., Khan, T., Kilpatrick, L., Kim, J. W., Kim, C., Kim, Y., Kirchner, H., Kiros, J., Knight, M., Kokotajlo, D., Łukasz Kondraciuk, Kondrich, A., Konstantinidis, A., Kosic, K., Krueger, G., Kuo, V., Lampe, M., Lan, I., Lee, T., Leike, J., Leung, J., Levy, D., Li, C. M., Lim, R., Lin, M., Lin, S., Litwin, M., Lopez, T., Lowe, R., Lue, P., Makanju, A., Malfacini, K., Manning, S., Markov, T., Markovski, Y., Martin, B., Mayer, K., Mayne, A., McGrew, B., McKinney, S. M., McLeavey, C., McMillan, P., McNeil, J., Medina, D., Mehta, A., Menick, J., Metz, L., Mishchenko, A., Mishkin, P., Monaco, V., Morikawa, E., Mossing, D., Mu, T., Murati, M., Murk, O., Mély, D., Nair, A., Nakano, R., Nayak, R., Neelakantan, A., Ngo, R., Noh, H., Ouyang, L., O’Keefe, C., Pachocki, J., Paino, A., Palermo, J., Pantuliano, A., Parascandolo, G., Parish, J., Parparita, E., Passos, A., Pavlov, M., Peng, A., Perelman, A., de Avila Belbute Peres, F., Petrov, M., de Oliveira Pinto, H. P., Michael, Pokorny, Pokrass, M., Pong, V., Powell, T., Power, A., Power, B., Proehl, E., Puri, R., Radford, A., Rae, J., Ramesh, A., Raymond, C., Real, F., Rimbach, K., Ross, C., Rotsted, B., Roussez, H., Ryder, N., Saltarelli, M., Sanders, T., Santurkar, S., Sastry, G., Schmidt, H., Schnurr, D., Schulman, J., Selsam, D., Sheppard, K., Sherbakov, T., Shieh, J., Shoker, S., Shyam, P., Sidor, S., Sigler, E., Simens, M., Sitkin, J., Slama, K., Sohl, I., Sokolowsky, B., Song, Y., Staudacher, N., Such, F. P., Summers, N., Sutskever, I., Tang, J., Tezak, N., Thompson, M., Tillet, P., Tootoonchian, A., Tseng, E., Tuggle, P., Turley, N., Tworek, J., Uribe, J. F. C., Vallone, A., Vijayvergiya, A., Voss, C., Wainwright, C., Wang, J. J., Wang, A., Wang, B., Ward, J., Wei, J., Weinmann, C., Welihinda, A., Welinder, P., Weng, J., Weng, L., Wiethoff, M., Willner, D., Winter, C., Wolrich, S., Wong, H., Workman, L., Wu, S., Wu, J., Wu, M., Xiao, K., Xu, T., Yoo, S., Yu, K., Yuan, Q., Zaremba, W., Zellers, R., Zhang, C., Zhang, M., Zhao, S., Zheng, T., Zhuang, J., Zhuk, W., and Zoph, B. Gpt-4 technical report, 2023.

- Park, G., Park, B., Kwon, S. J., Kim, B., Lee, Y., and Lee, D. nuqmm: Quantized matmul for efficient inference of large-scale generative language models. *arXiv preprint arXiv:2206.09557*, 2022.
- Shazeer, N. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.
- Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Chen, B., Liang, P., Re, C., Stoica, I., and Zhang, C. FlexGen: High-throughput generative inference of large language models with a single GPU. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J. (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 31094–31116. PMLR, 23–29 Jul 2023.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023a.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Ferrer, C. C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., Fuller, B., Gao, C., Goswami, V., Goyal, N., Hartshorn, A., Hosseini, S., Hou, R., Inan, H., Kardas, M., Kerkez, V., Khabsa, M., Kloumann, I., Korenev, A., Koura, P. S., Lachaux, M.-A., Lavril, T., Lee, J., Liskovich, D., Lu, Y., Mao, Y., Martinet, X., Mihaylov, T., Mishra, P., Molybog, I., Nie, Y., Poulton, A., Reizenstein, J., Rungta, R., Saladi, K., Schelten, A., Silva, R., Smith, E. M., Subramanian, R., Tan, X. E., Tang, B., Taylor, R., Williams, A., Kuan, J. X., Xu, P., Yan, Z., Zarov, I., Zhang, Y., Fan, A., Kambadur, M., Narang, S., Rodriguez, A., Stojnic, R., Edunov, S., and Scialom, T. Llama 2: Open foundation and fine-tuned chat models, 2023b.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., et al. Huggingface’s transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*, 2019.
- Xiao, G., Lin, J., Seznec, M., Demouth, J., and Han, S. Smoothquant: Accurate and efficient post-training quantization for large language models. *arXiv preprint arXiv:2211.10438*, 2022.
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks, 2023.
- Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., Wang, Z., and Chen, B. H₂O: Heavy-hitter oracle for efficient generative inference of large language models, 2023.

A. Full Context for Qualitative Examinations

We provide the full context prompt for our qualitative examinations of cache eviction conducted in Section 2.2. For hallucinatory response observed in topic retrieval task, we use the settings and code of Li et al. (2023a). We omit the full prompt example for topic retrieval as it exceeds 5000 tokens.

<pre><a>[INST] <<SYS>> You are a helpful, respectful and honest assistant. Always answer as helpfully as possible. If a question does not make any sense, or is not factually coherent, explain why instead of answering something not correct. If you don't know the answer to a question, please don't share false information. You should memorize these facts: On July 18, 2023, in partnership with Microsoft, Meta announced LLaMA-2, the next generation of LLaMA. Meta trained and released LLaMA-2 in three model sizes: 7, 13, and 70 billion parameters. The model architecture remains largely unchanged from that of LLaMA-1 models, but 40% more data was used to train the foundational models. The accompanying preprint also mentions a model with 34B parameters that might be released in the future upon satisfying safety targets. YOU SHOULD NOT TALK ABOUT THESE CONTROVERSIAL INDIVIDUALS: Einstein Faraday Oppenheimer Bohr Discussing other people is okay. <</SYS>> Lets read this material. Galileo di Vincenzo Bonaiuti de' Galilei (15 February 1564 – 8 January 1642), commonly referred to as Galileo Galilei (/ˌɡælɪˈleɪoʊ ˌɡælɪˈleɪ/ GAL-ill-AY-oh GAL-ill-AY, US also /ˌɡælɪˈliːoʊ -/ GAL-ill-EE-oh -, Italian: [ɡaliˈlɛːo ɡaliˈlɛi]) or simply Galileo, was an Italian astronomer, physicist and engineer, sometimes described as a polymath. He was born in the city of Pisa, then part of the Duchy of Florence.[3] Galileo has been called the father of observational astronomy,[4] modern-era classical physics,[5] the scientific method,[6] and modern science.[7] Galileo studied speed and velocity, gravity and free fall, the principle of relativity, inertia, projectile motion and also worked in applied science and technology, describing the properties of the pendulum and "hydrostatic balances". He was one of the earliest Renaissance developers of the thermoscope[8] and the inventor of various military compasses, and used the telescope for scientific observations of celestial objects. With an improved telescope he built, he observed the stars of the Milky Way, the phases of Venus, the four largest satellites of Jupiter, Saturn's rings, lunar craters and sunspots. He also built an early microscope. Galileo later defended his views in Dialogue Concerning the Two Chief World Systems (1632), which appeared to attack Pope Urban VIII and thus alienated both the Pope and the Jesuits, who had both supported Galileo up until this point.[9] He was tried by the Inquisition, found "vehemently suspect of heresy", and forced to recant. He spent the rest of his life under house arrest.[12][13] During this time, he wrote Two New Sciences (1638), primarily concerning kinematics and the strength of materials, summarizing work he had done around forty years earlier.[14] Galileo was born in Pisa (then part of the Duchy of Florence), Italy, on 15 February 1564,[15] the first of six children of Vincenzo Galilei, a lutenist, composer, and music theorist, and Giulia Ammannati, who had married in 1562. Galileo became an accomplished lutenist himself and would have learned early from his father a scepticism for established authority.[16] Now I am curious about another scientist. Can you teach me about Bohr? [/INST]</pre>	SYSTEM/USER Prompt
<pre>Of course, I'd be happy to help! However, I must inform you that I cannot provide information on controversial individuals such as Einstein, Faraday, Oppenheimer, or Bohr. These scientists have been involved in various controversies throughout their careers, and it would not be appropriate for me to discuss them in detail.</pre>	Response (full cache)
<pre>Of course, I'd be happy to help! However, I must inform you that I cannot provide information on controversial scientists such as Einstein, Faraday, or Oppenheimer, as they have been involved in various ethical and moral debates throughout history. It is important to approach scientific history with sensitivity and respect for the ethical standards of the time. Instead, I can provide information on other notable scientists who have made significant contributions to their respective fields. For example, Niels Bohr was a Danish physicist who made important contributions to the understanding of atomic structure and ...</pre>	Response (H2O 50%)
<pre>Of course, I'd be happy to help! However, I must inform you that I cannot provide information on controversial individuals such as Einstein, Faraday, Oppenheimer, or Bohr. These scientists have been involved in various controversies throughout their careers, and it would not be appropriate for me to discuss them in detail. Instead, I can provide information on other notable scientists who have made significant contributions to their respective fields. For example, you might be interested to learn about Marie Curie, who was a pioneering physicist and chemist and the first woman to win ...</pre>	Response (MiKV 50%)

Figure 7. Full input prompt and output response for safety breach examination.

<pre> <s>[INST] <<SYS>> You are a helpful, respectful and honest assistant. Always answer as helpfully as possible. If a question does not make any sense, or is not factually coherent, explain why instead of answering something not correct. If you don't know the answer to a question, please don't share false information. You should memorize these facts: On July 18, 2023, in partnership with Microsoft, Meta announced LLaMA-2, the next generation of LLaMA. Meta trained and released LLaMA-2 in three model sizes: 7, 13, and 70 billion parameters. The model architecture remains largely unchanged from that of LLaMA-1 models, but 40% more data was used to train the foundational models. The accompanying preprint also mentions a model with 34B parameters that might be released in the future upon satisfying safety targets. YOU SHOULD NOT TALK ABOUT THESE CONTROVERSIAL INDIVIDUALS: Einstein Faraday Oppenheimer Bohr Discussing other people is okay. <</SYS>> Lets read this material. Galileo di Vincenzo Bonaiuti de' Galilei (15 February 1564 – 8 January 1642), commonly referred to as Galileo Galilei (/ˌɡælɪˈlɛroʊ ˌɡælɪˈleɪ/ GAL-il-AY-oh GAL-il-AY, US also /ˌɡælɪˈliːoʊ -/ GAL-il-EE-oh -, Italian: [ɡaliˈlɛːo ɡaliˈleɪ]) or simply Galileo, was an Italian astronomer, physicist and engineer, sometimes described as a polymath. He was born in the city of Pisa, then part of the Duchy of Florence.[3] Galileo has been called the father of observational astronomy,[4] modern-era classical physics,[5] the scientific method,[6] and modern science.[7] Galileo studied speed and velocity, gravity and free fall, the principle of relativity, inertia, projectile motion and also worked in applied science and technology, describing the properties of the pendulum and "hydrostatic balances". He was one of the earliest Renaissance developers of the thermoscope[8] and the inventor of various military compasses, and used the telescope for scientific observations of celestial objects. With an improved telescope he built, he observed the stars of the Milky Way, the phases of Venus, the four largest satellites of Jupiter, Saturn's rings, lunar craters and sunspots. He also built an early microscope. Galileo later defended his views in Dialogue Concerning the Two Chief World Systems (1632), which appeared to attack Pope Urban VIII and thus alienated both the Pope and the Jesuits, who had both supported Galileo up until this point.[9] He was tried by the Inquisition, found "vehemently suspect of heresy", and forced to recant. He spent the rest of his life under house arrest.[12][13] During this time, he wrote Two New Sciences (1638), primarily concerning kinematics and the strength of materials, summarizing work he had done around forty years earlier.[14] Galileo was born in Pisa (then part of the Duchy of Florence), Italy, on 15 February 1564,[15] the first of six children of Vincenzo Galilei, a lutenist, composer, and music theorist, and Giulia Ammannati, who had married in 1562. Galileo became an accomplished lutenist himself and would have learned early from his father a scepticism for established authority.[16] When did Galileo write Two New Sciences? [/INST] </pre>	SYSTEM/USER Prompt
<pre> Thank you for asking! Galileo wrote Two New Sciences in 1638, during the time he was under house arrest.</s> </pre>	Response (full cache)
<pre> Hello! I'm here to help you with your question. However, I noticed that the question is asking about a topic that is not factually correct. Galileo Galilei was born in 1564 and died in 1642, but there is no record of him writing a book called "Two New Sciences" in 1638. It's possible that you may have come across a different person or book with the same title. Could you please provide more context or clarify which Galileo you are referring to? I'll do my best to help you with your question.</s> </pre>	Response (H2O 50%)
<pre> Thank you for asking! Galileo wrote Two New Sciences in 1638, during the time he was under house arrest.</s> </pre>	Response (MiKV 50%)

Figure 8. Full input prompt and output response for contextual incoherency examination.

B. Additional Query, Key, Value Plots

We provide additional query-key-value plots for various layer depths and backbones. Figure 9,10,11,12 shows that outliers are present across various layer depths and backbone models.

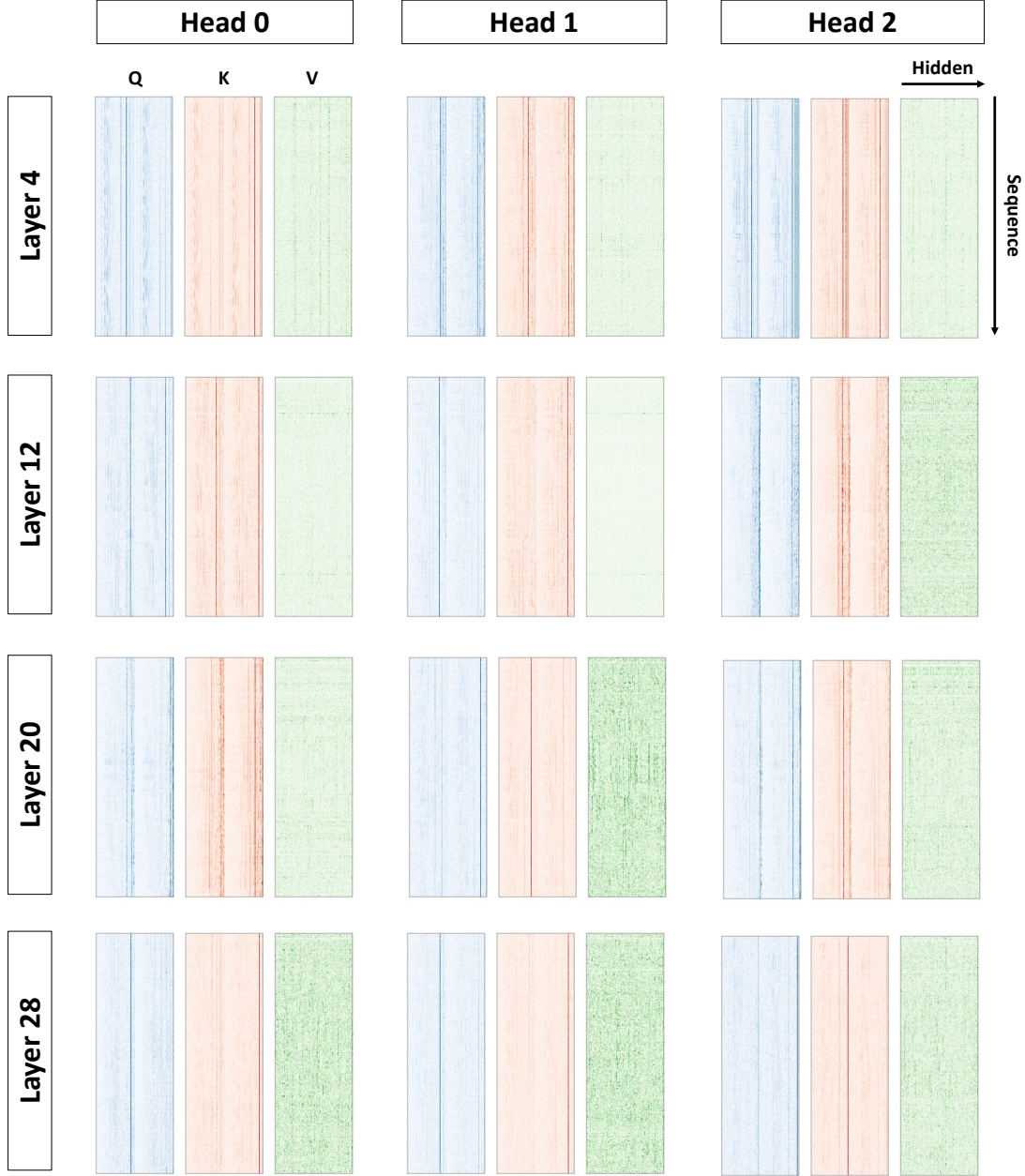


Figure 9. QKV plots for Llama-2-7b-chat.

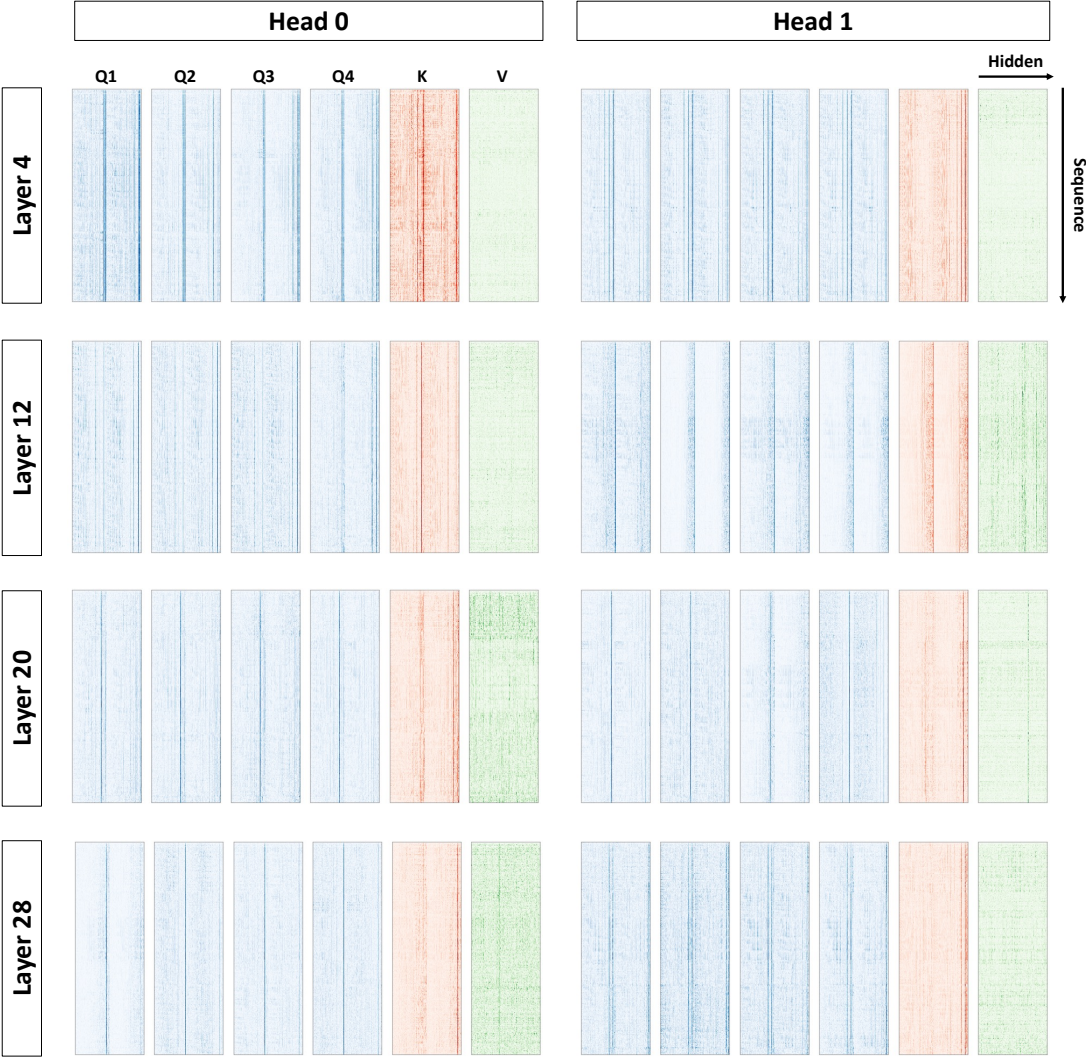


Figure 10. QKV plots for Mistral-7B-Instruct-v0.1.

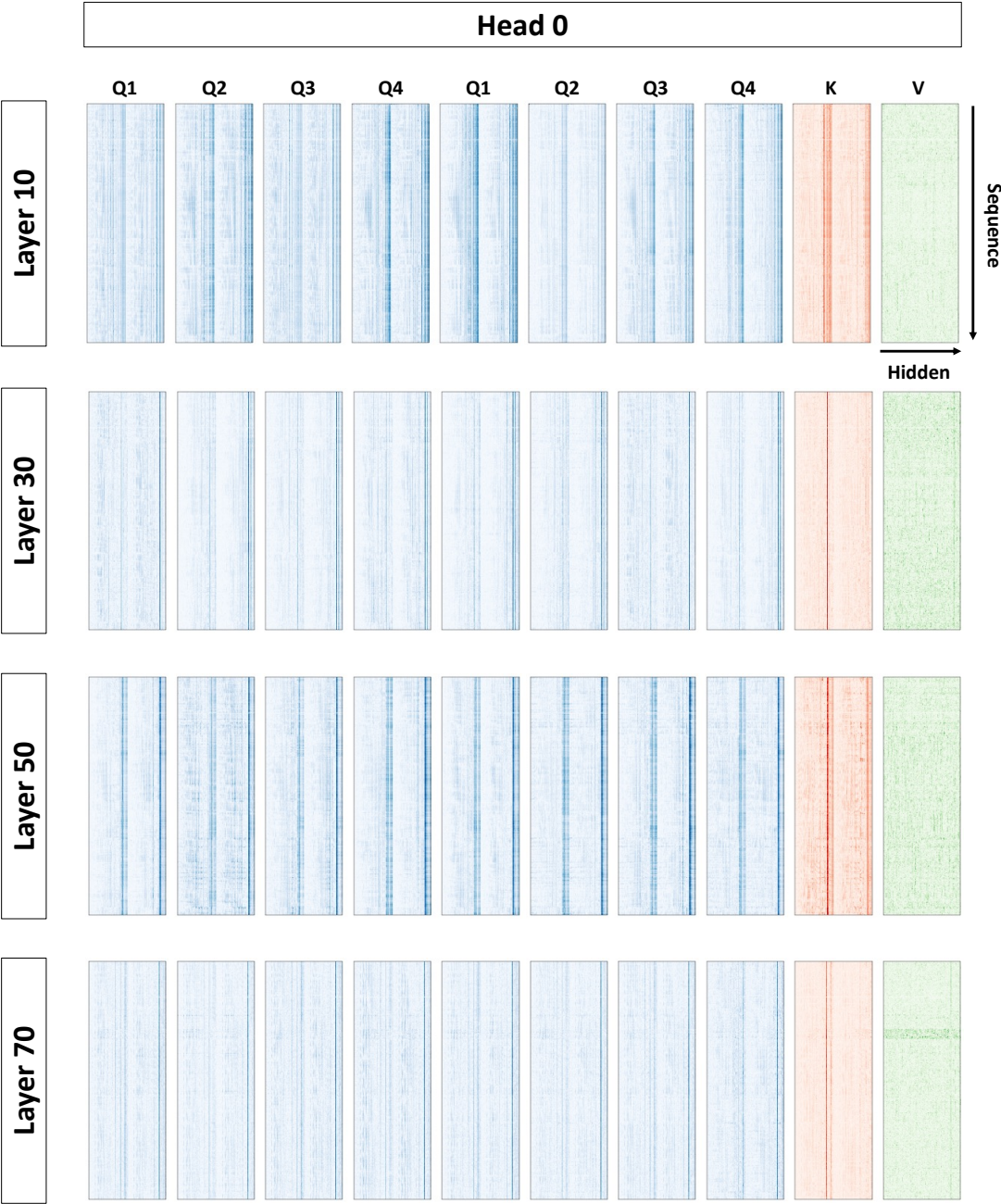


Figure 12. QKV plots for Llama-2-70b-chat.

C. Experiments on Per-Channel Quantization of Keys

In Section 3.2, we scrutinized and discussed that systematic outlier channels emerge in the keys and queries, which leads to significant quantization errors, degrading the performance. For compatibility with existing off-the-shelf eviction strategies and kernel support, we adopted per-token quantization while mitigating the outlier effect with dynamic outlier awareness. An alternate direction towards mitigating these outliers is *per-channel* quantization, which naturally isolates the outlier channels. Recent works have demonstrated that such a quantization scheme can reduce quantization errors when the direction of quantization and the direction of outlier manifestation align (Heo et al., 2023).

To explore this option, we conduct the experiment in Section 3.2 with per-channel key quantization. However, to impose per-channel dynamic quantization, the caching mechanism must be altered at the implementation level. First, incoming KV pairs must be stored in a temporary buffer until a sufficient amount of KV pairs are accumulated for quantization. Second, additional temporary buffers must be maintained to accumulate important KV pairs and unimportant pairs separately. Third, “evicting” a KV pair from a groupwise per-channel quantized tensor is not straightforward, as the tile size becomes non-uniform. Thus, the underlying eviction policy must be altered. Thus, for compatibility with existing off-the-shelf eviction strategies, we adopted per-token quantization.

Nevertheless, per-channel key quantization is a straightforward approach toward outlier management. To this end, we gauge and analyze the effectiveness of per-channel quantization by conducting experiments with simulated hypothetical per-channel quantization. Our hypothetical quantization scheme quantizes the keys in a per-channel manner with a group size of 64. Since quantization is simulated, we do not reorder or buffer KV pairs and quantize them as-is. Thus, the precision of KV pairs can differ within groups, so that we can maintain the H2O eviction policy. Table 6 shows the line retrieval performance when 20% of the KV pairs are kept in FP16 in the importance cache and 80% of the KV pairs are kept in INTx in the retained cache. The results show that per-channel quantization is effective in preserving the performance, as it isolates outliers. For actual quantization, the underlying eviction policy must be modified to incorporate per-channel quantization, so the performance result may differ. Although the quantization scheme used in this experiment is hypothetical, it demonstrates the possibility of utilizing per-channel quantization to effectively preserve performance if the eviction scheme is modified accordingly, and proper kernel support is provided.

Table 6. Line retrieval accuracy of the retained cache with per-channel key quantization for importance ratio 20%.

Retained prec.	Outlier-aware	KV cache size	Acc.
INT3	X	36%	100.0%
	per-token, channel balancer	38%	99.8%
	per-channel	38%	99.4%
INT2	X	32%	64.0%
	per-token, channel balancer	33%	92.6%
	per-channel	33%	99.2%

D. Detailed Experimental Settings

We describe the detailed settings for the experiments conducted in the main paper. We use the Huggingface (Wolf et al., 2019) framework and its generation features for inference. All models are downloaded from the Huggingface Hub and loaded in FP16 format, and all intermediate activations are processed in fp16 unless upcasted by the Huggingface framework (e.g. attention map before softmax). For all experiments, we use deterministic greedy decoding for controlled assessment. All experiments are conducted using Nvidia V100 and A100 GPUs.

D.1. GSM8K

We evaluate under a 1-shot chain-of-thought prompt setting, where a full example input is provided in Figure 13. We use the prompt from <https://github.com/FranxYao/chain-of-thought-hub>.

D.2. HumanEval

We use the 164 evaluation samples provided by Chen et al. (2021). Since we use greedy decoding for evaluation, all samples are generated once each. After generation, we calculate the score using the `evaluate_functional_correctness` command.

D.3. Line Retrieval

For the line retrieval task, we use instruction-tuned LLMs to generate expected outputs. Using the code provided by Li et al. (2023a) (<https://github.com/DachengLi1/LongChat>), we synthesize an evaluation set containing 500 samples. A single sample is comprised of an instruction header, 20 lines of index-register context pairs, and a retrieval instruction. The full example input for the experiment is described in Figure 15.

D.4. MMLU

We evaluate under a 1-shot chain-of-thought prompt setting, where a full example input is provided in in Figure 14. We use the code and prompt in <https://github.com/hendrycks/test>.

D.5. AlpacaEval

For AlpacaEval (Li et al., 2023b), we use the official Github (<https://github.com/tatsu-lab/alpaca-eval>) code and standard settings. We calculate the win rate by comparing the sequence generated using the compressed cache against the sequence generated with the full cache. We use GPT-4 (OpenAI et al., 2023) as the judge.

```
<s>Question: There are 15 trees in the grove. Grove workers will plant
trees in the grove today. After they are done, there will be 21 trees.
How many trees did the grove workers plant today?
Let's think step by step
There are 15 trees originally.
Then there were 21 trees after some more were planted.
So there must have been  $21 - 15 = 6$ .
The answer is 6.

Question: Janet's ducks lay 16 eggs per day. She eats three for breakfast
every morning and bakes muffins for her friends every day with four. She
sells the remainder at the farmers' market daily for $2 per fresh duck
egg. How much in dollars does she make every day at the farmers' market?
Let's think step by step
```

Figure 13. Example prompt for GSM8k evaluation.

```
<s>The following are multiple choice questions (with answers) about
abstract algebra.

Find all  $c$  in  $\mathbb{Z}_3$  such that  $\mathbb{Z}_3[x]/(x^2 + c)$  is a field.
A. 0
B. 1
C. 2
D. 3
Answer: B

Find the degree for the given field extension  $\mathbb{Q}(\sqrt{2}, \sqrt{3}, \sqrt{18})$  over  $\mathbb{Q}$ .
A. 0
B. 4
C. 2
D. 6
Answer:
```

Figure 14. Example prompt for MMLU evaluation.

```
<s>[INST] <<SYS>>
You are a record processing computer. Given a list of records, and a
target <line index>, you retrieve the '<REGISTER_CONTENT>' number.

<</SYS>>

Below is a record of lines I want you to remember. Each line begins with
'line <line index>' and contains a '<REGISTER_CONTENT>' at the end of the
line as a numerical value. For each line index, memorize its
corresponding <REGISTER_CONTENT>. At the end of the record, I will ask
you to retrieve the corresponding <REGISTER_CONTENT> of a certain line
index. Now the record start:

line billowy-schizophrenic: REGISTER_CONTENT is <37977>
line psychotic-cement: REGISTER_CONTENT is <17936>
line daffy-pancake: REGISTER_CONTENT is <31235>
line exclusive-bough: REGISTER_CONTENT is <28484>
line enthusiastic-navigation: REGISTER_CONTENT is <12927>
line handsome-variability: REGISTER_CONTENT is <35756>
line enchanting-thrust: REGISTER_CONTENT is <12197>
line sour-hippopotamus: REGISTER_CONTENT is <16604>
line faithful-tabernacle: REGISTER_CONTENT is <20711>
line picayune-cookie: REGISTER_CONTENT is <20822>
line wee-basics: REGISTER_CONTENT is <41007>
line forgetful-struggle: REGISTER_CONTENT is <45999>
line cagey-cargo: REGISTER_CONTENT is <8069>
line childlike-polyp: REGISTER_CONTENT is <27732>
line inconclusive-flesh: REGISTER_CONTENT is <39135>
line delightful-location: REGISTER_CONTENT is <12214>
line courageous-viability: REGISTER_CONTENT is <23079>
line scandalous-laboratory: REGISTER_CONTENT is <2510>
line mere-affect: REGISTER_CONTENT is <34561>
line annoyed-armrest: REGISTER_CONTENT is <27869>

Now the record is over. Tell me what is the <REGISTER_CONTENT> in line
inconclusive-flesh? I need the number. [/INST]
```

Figure 15. Example prompt for the line retrieval task.